Chibi claymation SDF prototype plan

Prepared for Charles | 19 October 2026 | Revised scope and implementation checklist

Recommendation

Build a small clay playground using Chibi's existing SDF model and material infrastructure as the starting point, while extending it wherever the intended quality, authoring, interoperability, or performance requires. Reuse is a baseline, not a ceiling. Start with the current Model Preview for look development, then prove free-camera 3D rendering, depth, materials, shadows, picking, and explicitly chosen collision proxies. Add bounded OpenVDB level-set interchange as the recommended professional asset bridge. Finish four focused scenes and one short interactive demonstration before taking on persistent sculpting or fluid simulation.

This is a plan for implementation review. The source inspection establishes what code exists; it does not establish runtime correctness, performance, or that the unfinished self-hosting work has passed. All budgets below are proposed acceptance targets, not measurements or promises. This revision covers the plan and its PDF for publication on a dedicated documentation branch. It does not start implementation or change the active self-hosting branch.

The recommended order is:

1. Establish a reproducible baseline and agree on authoring syntax and visual direction.
2. Validate and extend the SDF contract, add a bounded OpenVDB asset bridge, and connect SDF models to the separate 3D scene renderer.
3. Build the clay material, stable surface coordinates, and stepped presentation timing.
4. Exercise geometry, animation, and physics in small repeatable scenes.
5. Measure and optimize the entire slice, including compile and iteration time.
6. Design a Chibi shader-language frontend that reuses the validated compiler path.
7. Evaluate NanoVDB runtime storage and optional USD scene interchange against concrete workloads; keep sculpting and solver experiments separately gated.

Scope choices

Option	Deliverable	What it establishes	Main cost	Recommendation
A Material laboratory	Existing preview, calibration objects, controlled lighting, comparison toggles	Whether the clay look works	Lowest; does not establish scene3d integration	First checkpoint
B Clay pla yground	Four scenes, free camera, mixed geometry, responsive input, stepped poses, basic physics, bounded VDB interchange	Whether SDF clay is useful for Chibi game development	Moderate; adds rendering, ECS and interchange tooling	Main prototype
C Sculpt and interact	Small editable clay block, stamp and cut tools, one controllable body	Persistent local edits and collision updates	Higher; introduces a mutable representation and update scheduling	Optional follow-on
D Clay and fluid simulation	Deformable body and/or small fluid basin	A simulation research direction	Highest; requires solver, surface representation, synchronization, and new validation	Separate experiment

A Claybook-like demonstration is a useful direction for C or D, but its production architecture is much larger than a material or raymarching prototype. The first release should prove a compact, reusable engine capability.

What is already in Chibi

The read-only audit pinned main at 07eb2ab and codex/toolchain-cleanup-20261017 at dd2a100. At inspection the cleanup branch was 91 commits ahead and 13 behind main. Main includes the merged language and typed-runtime redesign. These are separate snapshots, not interchangeable baselines.

Area	Verified present	Boundary for this prototype
Constructive SDF models	Primitive and CSG graphs, transforms, smooth joins, parts, parameters, profiles, sweeps, sampled assets, and bounded mesh baking	Validate and extend the subset used; replace a limitation when the evidence justifies it
Model compilation	Distance IR, separate hit/part resolution, bounds, parameters, and dependencies	Geometry graph values are compiler descriptions, not ECS entities or sampled distances
SDF rendering	Orbitable Model Preview and planar/2.5D material path; hit-depth writes, analytic normals with fallbacks	Free-camera scene3d still draws primitives/meshes rather than model SDFs
Material integration	Part selection and material evaluation after the hit	Smooth union chooses the dominant part material; it does not blend two material graphs
Scene3d materials	Metallic/roughness GGX lighting, Normal Map and Bump operations	No separate specular, IOR, oiliness, clearcoat, second-lobe, or scattering output
Shader compiler	KDL graphs and small expressions to typed expression IR, CPU evaluation, GLSL, and Cute compilation	No general Chibi GPU shader-source language
ECS and language	Engine-owned ECS, system query syntax, typed runtime, automatic storage growth	Prepare GPU parameters on the CPU; per-pixel shaders do not query the ECS
3D physics	Box3D with box, sphere, capsule, hull, and static triangle-mesh colliders	No SDF collider in the current contract; visual deformation does not deform collision
Self-hosting work	Main recording-host proof; cleanup branch has a bounded authored compiler increment	M5 remains incomplete; it is not a dependency of this prototype

The scene boundary is explicit in declarative SDF modeling and 3D rendering, and confirmed by scene3d.c. The planar path ignores Transform3D. Porting its hit logic into scene3d still requires world-space shading and HDR/depth integration; its planar material outputs cannot simply be copied unchanged. Correct SDF shadow casting also needs its own pass integration.

Reuse the material system

Current Surface Output includes Base Color, Emission, Roughness, Metallic, Normal, Opacity, Unlit, Height, and Occlusion. The scene3d shader uses GGX, Smith visibility, Schlick Fresnel, and fixed dielectric F0 of 0.04; its roughness mapping uses alpha = max(roughness squared, 0.0025). Existing Bump and Normal Map operations evaluate detail at the resolved hit instead of every march sample. They affect shading rather than silhouette or collision. See the output contract, scene3d lighting, and surface detail implementation.

Reuse that single-lobe material with Metallic=0 for the first clay preset. An initial artist-facing Oiliness control should map to existing roughness/detail settings; it is not independent dielectric reflectance. Independent specular/IOR, clearcoat, another reflection lobe, and scattering need separately scoped renderer changes. The planar SDF path maps roughness to HRC gloss, so final material approval must happen under scene3d lighting.

Existing interchange and sampled storage

Chibi already has a triangulated OBJ to sampled-SDF converter, using an RG16 PNG atlas with explicit Half Size, Resolution, Range, and Lipschitz metadata. It has bounded input/work limits and recorded conversion provenance. It also has glTF 2.0 and GLB model import, including mesh/material conversion. Those are useful integration points, not a complete professional volume pipeline. The older converter README's test commands are historical; use current admitted selectors and the current test policy.

Keep authored analytic models and imported source assets intact. An atlas, a NanoVDB buffer, or a mesh bake is a derived runtime representation that can be regenerated. Extending precision, resource access, traversal, bounds, materials, or the field compiler is acceptable when needed; preserve explicit semantics and test the replacement rather than forcing every feature into today's storage limits.

Existing implementation budgets

The audited source declares 1,024 shader IR nodes, 32 geometry parameter slots including private derived values, 32 model material slots, 32 combined material parameter slots, eight image/settings variants per composed program, and 512 ramp vec4 resources. The march loop has a 128-step limit, or 512 for the sampled/Lipschitz path. These are implementation limits, not demonstrated scene capacities. See shader program contracts, model program contracts, and the SDF fragment shader.

Real current authoring examples

The checked-in SurfaceSdf fixture uses this component structure:

```
Geometry3D depth=24.0 elevation=4.0 height=8.0 width=24.0
SdfSurface3D enabled=#true
Shader material="SurfaceBands" shape="SurfaceRing"
Transform x=-22.0 y=-18.0
```

This is an existing raw-shape example in the planar path, not the proposed free-camera demo. Shader.shape can also bind a declarative model. Models calculate their own bounds; Geometry3D dimensions do not resize them. The constructive model graph defines geometry and part attribution. A separate material graph defines appearance after intersection.

Existing CPU/ECS syntax includes:

```
FreeMotion :: system(position: Transform, velocity: Velocity)
    without Collider, Body {
    position += velocity * dt
}
```

That is a system example from the language reference, not shader syntax. The shader-library fixture ShaderMath.kdl and material PulseRing.kdl show the actual current graph-based GPU authoring.

Start from focused fixtures

Use scene3d Shapes.kdl for a small scene/light baseline, Rendering3D.kdl for coverage, and Physics3DProbe.chibi for the physics starting point. Keep SurfaceSdf as the old-path control. Add one new bounded clay model fixture: the examples policy says the separate examples were retired on October 17, and the audit found no checked-in project currently saving a declarative model.

Keep the cleanup branch's bounded compiler status independent. Its accepted authored subset does not include ECS, aggregates, general calls, or shaders, and it explicitly does not complete M5. Renderer and runtime shader files in the audited cleanup snapshot match main; a few graph/model files differ. No self-hosted compiler, LLVM migration, new ECS, or new math library is required for this slice.

What the references establish

Claybook's GDC 2018 presentation describes a mipmapped sampled world SDF, local volume updates, GPU simulation, and integration into Unreal Engine. Its deforming clay bodies use particle-based shape matching and generated triangle meshes; a slide explicitly says the tested bounding-box SDF raymarch path was not used for those particle-deformed bodies. Its fluid path uses SPH and rebuilds a separate SDF for rendering. These are related systems with distinct data and costs. They are not a single arbitrary distance function that supplies deformable physics for free. Read the GDC slides, PDF pages 7-9, 19-24, 44-54, and 58. The GDC session page is the verified video entry; the video itself was not watched for this plan.

For the material, Howell's Aardman/Bath work supports treating surface roughness and oiliness as separate artistic controls, paying attention to grazing-angle reflections, and adding believable worked-surface detail. The production thesis, chapter 4, printed pages 67-74 and 83-85 is the strongest material reference here. Rowbotham's shader report, pages 4-6, provides useful layered-reflection and fingerprint comparisons. Johansson's implementation article demonstrates texture, normal, roughness, and vertex-displacement experiments. Its randomized UV changes are an artistic option rather than the default proposed here.

The mathematical contract matters just as much as the look. An implicit function, an exact signed distance, and a conservative distance bound are different promises. Hart establishes distance-bound sphere tracing and derives bounds for composition and deformation. We should make those promises explicit instead of assuming every blend or warp preserves distance. See Hart, sections 2.1-2.2 and the transformation appendices.

Visual direction and clay controls

The target is a hand-shaped miniature world with coherent marks and deliberate animation. Start from one neutral material and one colored material. Judge them at gameplay scale and in motion, using identical exposure and lighting for comparisons.

Shape and silhouette

- Make the basic forms slightly asymmetric. Use softened joins, uneven edges, thumb depressions, compressed contact areas, and a few larger tool marks.
- Reserve true geometry or field deformation for changes visible in silhouette or contact shadow. Put fine fingerprints and scuffs primarily into shading normals and roughness.
- Keep large-form deformation, medium-scale surface variation, and microdetail independently adjustable. A single noise function should not control all three.
- Use purposeful variation. A smooth hero eye, a worked cheek, and a rough base need not share the same detail intensity.

Reflection and color

- Reuse the existing GGX metallic/roughness material with Metallic=0, broad highlights, and existing Bump/Normal Map detail. Compare a restrained second oily lobe only if the baseline cannot achieve the reference look, with a separate renderer change approved first.
- Expose understandable controls such as base color, surface roughness, oiliness, fingerprint strength and scale, and sculpt irregularity. Keep low-level BRDF controls in an advanced view.
- Treat roughness 0.45-0.60 and 0.60-0.80 as trial ranges for smoother/oilier and drier surfaces, respectively. Their meaning depends on Chibi's actual shading implementation; neither range is a reference-derived law.
- Keep pigment noise faint. Do not make fingerprints primarily dark color stripes. Avoid uniform high-frequency noise that reads as sand, stone, or orange peel.
- Add optional, restrained subsurface scattering only after the baseline passes. It needs model-scale and thickness controls, a cost measurement, and an off switch. Warm red scattering is an art choice rather than a universal clay property.

Surface attachment and time

- Give each instance stable material coordinates and a stable seed. Moving the object should not slide the texture through it.
- For deforming geometry, define whether marks follow rest space, an inverse deformation map, or a deliberately approximate mapping. Test it before adding more detail. At smooth joins, test changes in part ownership for coordinate or material popping; choose a stable model-space mapping where a part-local mapping cannot remain coherent.
- Change selected marks after a pose or contact event. Untouched areas and static scenery should remain still by default.
- Rotate or transform tangent-space normals with the corresponding basis. A UV rotation alone is insufficient; deriving the perturbation from a transformed height field is another option.

- Keep animation sampling separate from simulation and input. Proposed defaults are 60 Hz simulation and 12 visual poses per second, with 8, 12, 24, and smooth comparison modes. At 60 displayed frames per second, a 12-pose cadence holds each pose for five displayed frames.
- Prefer a smooth physical root with stepped local pose/deformation for the first playable test. A fully held root is a separate art option because it can make visible contact disagree with the collision body.

Lighting and presentation

Use a deliberate key, restrained fill, a readable background, and contact shadows. Include a raking-light inspection mode. Keep exposure fixed during comparison captures. Depth of field, grain, a tighter camera framing, and subtle surface boil are finishing options, each with an off switch. Establish the look without them first.

Required comparison controls

The laboratory should toggle implemented form irregularity, fingerprints, roughness variation, pose cadence, surface updates, shadows, and antialiasing separately. If an optional second reflection lobe or scattering experiment is approved, it also needs an isolated toggle; absent optional effects do not block the prototype. Save named presets and screenshots with their parameters so a pleasing result can be reproduced.

Architecture contracts

Keep four concepts separate

1. **Shape authoring:** reusable primitives, transforms, CSG operations, blends, and bounded deformations that define a surface.
2. **Rendering:** intersection, normal estimation, material evaluation, depth, lighting, and scene composition.
3. **Interaction:** picking, broad phase, collision proxies or field queries, contact response, and gameplay events.
4. **Persistent material state:** dents, stamps, cuts, paint, damage, and any volume or history used to retain edits.

A shape may share information across these systems without requiring identical representations. The first prototype should reuse the shape graph and bounds for rendering and picking, while using the existing supported collider forms where appropriate. Any visible/collision mismatch must be shown in a debug view and documented.

First class SDF support

The useful meaning of first class is an engine-recognized, inspectable resource that can be reused in models, entities, tools, and tests. It does not require adding a new language keyword immediately.

Proposed contract for each compiled field:

- Stable resource identity, version, parameter layout, source location, and meaningful diagnostics.
- An explicit sign convention and unit scale; use negative inside and positive outside unless existing code requires a documented compatibility boundary.

- A known bound or an explicit unbounded classification. Scene culling must not depend on guessed extents.
- Distance classification: exact distance, conservative bound, or visual-only estimate. Arbitrary callbacks must not silently claim the first two.
- Surface/material identity, local/rest coordinates, and an explicit join policy. Keep current dominant-part material selection in the first slice; blending two material graphs is optional later work.
- Defined transform behavior, normal calculation, and precision/error tolerances.
- A supported CPU query path when picking or physics needs it, or an explicit unsupported result. Avoid synchronous GPU readback as a default gameplay query.
- Separate topology/structure changes from instance-parameter changes so animation does not recompile the entire shader every frame.

Use internal arenas, pools, resource caches, and explicit engine lifetimes as needed. Ordinary game code should not have to pass an allocator into every field, model, material, or ECS operation. Allocation and GPU memory should remain observable through profiling.

Distance and deformation safety

Rigid transforms preserve distance. Uniform scaling needs the matching distance correction. General nonuniform transforms, smooth blends, displacement, and bend/twist operations require a proven conservative rule, an appropriate bound, or a clearly labeled approximation. A field can define an attractive zero surface while returning unsafe march distances.

For the supported operation set, propagate bounds through compilation and test them against independent reference sampling. Unknown deformations should be rejected from the safe path or placed behind an explicit experimental mode. Simply multiplying every step by an arbitrary safety factor is a diagnostic fallback, not a proof.

Use separate geometry and shading normals. Choose finite-difference spacing from scale/precision and compare against analytic derivatives where available. Handle near-zero gradients and CSG seams explicitly. Validate camera-inside cases, thin gaps, grazing rays, and contact offsets rather than relying only on front-facing spheres.

Scene rendering integration

The first scene3d task is a small proof with one SDF and one ordinary opaque object intersecting in screen space. It must establish camera-ray generation, coordinate conventions, near/far clipping, depth comparison and writes, viewport resizing, material lookup, and order-independent opaque composition.

Then add instance bounds and culling, shared field resources with instance parameters, deterministic material IDs, shadows, picking, and any required antialiasing path. Preserve the existing preview and ordinary rendering behavior with regressions. Do not assume that code working in an orbit preview already handles a perspective camera, mixed depth, or many instances.

If depth-aware composition is unavailable in the current backend, choose and document a supported route before writing a parallel renderer. Viable alternatives include a bounded per-object draw or a unified scene field pass. Their suitability must come from the actual backend and measurements.

ECS integration

Use the project's existing typed runtime and ECS conventions. Its ECS already grows storage automatically and does not require a user allocator for world creation. Candidate responsibilities are a field-resource handle, transform, material parameters, pose clock, render bounds, optional collider/proxy, and optional edit state. These are responsibilities, not a new approved naming scheme.

The render extraction step should gather stable handles and compact per-instance data. Simulation updates transforms and contact events; presentation samples poses; material events update only affected instances. Keep structural graph compilation out of ordinary per-frame entity iteration. Verify teardown, resource sharing, hot reload, and entity deletion rather than relying on sample-app lifetime.

Professional SDF interchange and storage roadmap

Recommendation: support a deliberately bounded OpenVDB level-set import/export profile first. Keep analytic CSG as the editable source for procedural Chibi models, and retain the original VDB for imported sampled assets. Evaluate NanoVDB as a compiled runtime snapshot when its memory or traversal benefits justify a backend extension. Add USD only when scene assembly or DCC exchange needs it. Continue using the existing glTF/GLB mesh pipeline. These choices solve different problems; none is a universally better SDF format.

What each representation is for

Representation	Recommended role	Important boundary
Chibi analytic CSG/model source	Procedural authoring, readable composition, live parameters, compiler-known bounds	Sampling cannot reconstruct the original operations, constraints, or material graph
OpenVDB .vdb	Professional sparse sampled-field interchange and offline processing	A grid can contain a level set, fog, vectors, or other data; the extension alone does not mean signed distance
NanoVDB snapshot	Candidate sparse CPU/GPU runtime cache	Static tree topology; changing values is different from adding/removing nodes or maintaining a level set
Existing RG16 atlas or another bounded dense cache	Simple reference runtime path and portability fallback	Quantization, sampling footprint, bounds and memory can make a different cache preferable
USD with UsdVol/OpenVDBAsset	Optional scene container referencing named field assets and placement	Does not replace the VDB payload, provide a solver, or guarantee that a renderer supports level-set surfaces
glTF/GLB mesh assets	Existing portable mesh/material import; mesh output after explicit tessellation	Core mesh primitives do not preserve an analytic CSG tree or the proposed sampled level-set contract

OpenVDB separates sparse data, transforms, and interpretation. A narrow-band level set stores distance near the surface and constant signed values outside that band. Those far-field constants are not globally exact distances; resampling or scaling may require distance reconstruction. See the

OpenVDB overview. Our import profile must validate the data it actually receives rather than trust a grid-class label.

NanoVDB uses a contiguous, pointerless layout and supports CPU/GPU access, but its tree topology is static. It is suitable for evaluating a prepared snapshot, not automatically a mutable sculpting store. Its portable PNanoVDB header provides C99/GLSL/HLSL entry points; Chibi must still prove buffer access, addressing, alignment, shader translation, and resource lifetime on each intended backend. See the NanoVDB FAQ. No CUDA requirement or automatic Cute integration is assumed.

USD's volume schema and OpenVDBAsset schema reference field files and selected grids. Scene composition, time samples, materials, and volume rendering support are separate compatibility questions. The USD Sdf namespace refers to scene description foundations, not signed-distance fields. The glTF specification describes the mesh bridge; any mesh-to-field conversion remains a separate, potentially lossy operation.

First supported VDB profile

Start with one explicitly selected float32 scalar level-set grid per static asset, uniform voxel spacing, rigid placement, an explicit unit conversion, and a negative-inside sign convention. Reject fog, unsigned fields, vector grids, nonfinite data, missing/ambiguous grids, unsupported versions/compression, and unsupported transforms with actionable errors. Nonuniform scale, shear, nonlinear transforms, sequences, multigrid materials, and editable sparse topology are later additions rather than silent approximations.

The first importer is shape-only with one explicit Chibi material binding. Preserve the original asset and a manifest recording source hash, grid name/class, converter and dependency versions, units, index-to-world transform, voxel spacing, active bounds, signed band/background values, precision, and the chosen interpolation/normal policy. Chibi material IDs or edit history, if added later, are separate engine metadata; do not promise VDB roundtrips of arbitrary shader networks.

Convert through an asset-build boundary into an engine-ready resource. Try the existing sampled atlas as the small correctness reference, but do not require it as the shipping representation. Recompute or validate the cache's Range, Lipschitz/error bounds, padding and border behavior after resampling and quantization. Trilinear samples and a level-set tag do not prove conservative sphere-tracing steps. If the required asset cannot fit the existing contract with acceptable quality, extend the cache, field/traversal contract, or backend and test that extension. Do not silently clamp or mislabel it as exact distance. For export, a valid zero surface or conservative step bound is not enough to label samples GRID_LEVEL_SET. Export admitted true-distance samples, or rebuild/re-distance the zero surface and verify near-band distance error; otherwise reject that export or use an explicitly separate generic-scalar profile.

Proposed first-profile limits: one static grid, at most 64 MiB compressed source, at most 16 million active voxels, and at most 512 MiB peak conversion memory. The initial dense comparison is capped at 128 cubed samples. These are separate provisional limits, not a promise that every asset meeting one limit fits the others. Account for sparse tree overhead, tiles, decompression and temporary buffers; reject before uncontrolled growth. Choose tighter admitted fixtures and a deadline below 60 seconds for tests. Larger production conversions require an explicit resource policy before support is claimed.

Dependencies, licensing, and C integration

Use an optional pinned external conversion tool at asset-build time so a shipped game need not include the full OpenVDB toolchain. The official vdb_tool and NanoVDB conversion tools are candidates to evaluate, not proof that an off-the-shelf command already emits Chibi's atlas. Prove the end-to-end adapter in I1 before committing to its packaging.

Chibi's repository policy keeps logic in C and enumerates narrow existing C++ shim exceptions. Do not introduce arbitrary C++ engine logic for VDB. Prefer official external conversion plus a C-readable boundary; a new authored C++ shim or tooling exception requires an explicit reviewed policy decision. Engine-owned allocation and resource management remain the default for users.

The inspected OpenVDB 13.0.0 license is Apache-2.0. Pin the chosen release and audit its actual dependency graph and notices. The core build configuration and build guide expose C++/CMake/TBB and option-dependent dependencies such as Boost iostreams, Imath, Blosc and Zlib. Disabling compression support can reject real DCC files; test the chosen profile rather than claiming all VDB files work. NanoVDB reader-only integration can be lightweight, but conversion and tools can bring those dependencies back.

OpenUSD is a separate optional SDK/tooling dependency. The inspected 26.08 license file names the Tomorrow Open Source Technology License 1.0 and includes third-party notices; do not label the whole distribution simply Apache-2.0. Record shipped components and applicable notices for every selected package. This is a dependency-review checklist, not a blanket licensing clearance.

Support decisions and promotion gates

1. **Recommend for the prototype roadmap:** bounded VDB import/export plus one externally authored sculpt in the laboratory and 3D scene. I1 can run alongside renderer work after G1. Professional interchange is accepted only when GI1 passes; an earlier visual demo may still be shown with that gate explicitly pending.
2. **Measure before adopting NanoVDB at runtime:** compare against the dense reference at matched error/quality. Record compressed bytes, CPU resident and peak conversion memory, GPU bytes, upload time, lookup/render cost and rebuild cost. Adopt it only if the workload benefit pays for backend and maintenance complexity; retain a measured fallback or declare platform limits.
3. **Optional USD integration:** justify with a real scene-exchange use case. Test one referenced VDB plus one mesh, units/axes, placement, selected grid, missing dependencies, resolver/path safety and renderer compatibility. Defer arbitrary composition, animation sequences and material-network translation until separately scoped.
4. **Optional mesh/field extensions:** improve existing glTF/OBJ flows only where the test corpus exposes a gap. Require watertight/sign policy and explicit repair choices for voxelization; require geometric-error and material-loss reporting for tessellation. No lossless analytic roundtrip claim.
5. **Optional mutable storage or simulation:** choose an edit store and a solver independently. A VDB file, NanoVDB buffer, USD scene, raymarcher, or extracted mesh does not supply plasticity, contact response, or fluid dynamics.

Authoring and shader language decisions

Approve the authoring examples before implementation. Use the real current syntax as the compatibility baseline; the names below describe options rather than a new grammar.

Authoring choices

Option	Benefit	Risk	Suggested use
Existing declarative model syntax and standard functions	Lowest migration risk; current tools and diagnostics	Some experiments may need new operations	Default prototype authoring
Typed SDF value with composable library operations	Readable reuse in ordinary game code; engine can inspect structure	Requires clear value, lifetime, and dynamic-parameter semantics	Candidate first class API evolution
Dedicated field block or declaration	Strong domain diagnostics and concise models	More grammar and tooling; may duplicate existing constructs	Discuss only if real examples justify it
Raw shader escape hatch	Fast expert experiments	Weak portability, validation, and CPU parity	Internal research boundary, not the normal game API

Compare each on the same three examples: a rounded toy with a cutout, an articulated squashable character, and a parameterized material with fingerprints. Include construction, reuse on two entities, a runtime parameter change, a compile error, and cleanup. The best option should be readable without exposing renderer plumbing or mandatory allocators.

Recommended compiler direction

Keep shape structure and material calculations as distinct domains with explicit interfaces. They may share typed operations and lowering utilities, but they should not be collapsed merely because both ultimately run on the GPU.

The intended long-term flow is:

- Existing model/graph authoring and a future Chibi shader frontend produce validated typed representations.
- Shape lowering preserves distance guarantees, bounds, transforms, material identity, and coordinate mapping.
- Material lowering preserves value types, resource bindings, stage restrictions, and parameter layouts.
- A shared backend boundary emits the supported shader source and binding metadata for Cute, with deterministic cache keys and diagnostics tied to source locations.
- CPU evaluation, when supported, consumes the same semantic operations with documented precision differences. It does not promise bitwise parity with the GPU.

Start the shader-language experiment with an expression-only GPU-safe subset: explicit float32 scalar/vector math already supported by the IR, constants, parameters, and acyclic pure helpers inlined into those operations. The existing IR is a straight-line expression graph. Reject loops and source-level control flow initially. Allow existing fixed image-sampling operations only if the chosen example needs them; defer new buffer/resource models and stage features. Reject heap allocation, runtime object graphs, unrestricted recursion, host I/O, unsupported dynamic dispatch, and unsupported stage operations with useful messages.

Do not make the clay prototype depend on compiling arbitrary Chibi programs for the GPU. First prove one material and one compiler-known analytic field operation can enter the existing pipeline from a typed source representation and produce equivalent results to their existing graph versions. The field example must retain its proven bounds and distance contract; an arbitrary float-returning function does not acquire those guarantees automatically. Material-function success alone does not establish constructive-geometry authoring. A general shader language needs its own reviewed design and compatibility plan. The preferred candidate is ordinary Chibi expression/function syntax inside an explicit shader-domain boundary. Compare that with dedicated shader declarations before choosing spellings. For graph/source interoperability, choose one editable source of truth rather than promising lossless bidirectional editing.

Compiler acceptance questions

- What is compile-time structure, what is a specialization, and what is an instance parameter?
- How are uniforms, textures, samplers, and buffer layouts reflected to the runtime?
- Which operations work on CPU, fragment GPU, and any future compute backend?
- How do errors identify the originating model operation or source expression?
- Which optimizations are legal for exact-distance and conservative-bound semantics?
- How are generated source, compiler versions, platform capabilities, and cache invalidation recorded?
- What happens on unsupported hardware or a shader compile failure?

Useful engine-aware optimizations include constant folding, common-subexpression elimination, conservative bound propagation, reuse of static graph code across instances, parameter-only updates, bounded specialization, and culling before expensive field evaluation. Introduce each with before/after measurements and semantic regressions.

Prototype scenes

Each scene should be selectable from the same demo, resettable with one action, seeded, and recordable using a fixed camera path. Include a compact metrics overlay and a plain preset file.

Scene 1 Material and geometry laboratory

Use a sphere, capsule, rounded block, a smooth join, a sharp cut, and one intentionally irregular sculpt, including the external VDB asset from I1. Show identical material variants under neutral, raking, and grazing light. Include a close-up and gameplay-distance view.

Pass: the chosen preset reads as worked clay in a still image and in a moving camera; disabling each effect explains its contribution; fingerprints retain plausible scale and attachment. No scattering or postprocessing is required to hide a broken baseline.

Scene 2 Miniature scene and depth test

Build a small tabletop set with a ramp, arch, hole, steps, and three differently colored props. Mix an ordinary opaque primitive with analytic SDF objects and the imported VDB sculpt. Orbit and move a perspective camera through the scene, including behind the arch and close to the ground.

Pass: correct mutual occlusion, stable depth, correct clipping and picking, readable contact shadows, no view-dependent disappearing geometry, and consistent material selection at joins.

Scene 3 Held pose character

Use an original simple character or toy, with a short reach, squash, recovery, and jump loop. Compare smooth animation with 8, 12, and 24 poses per second. Trigger a few local surface changes on selected pose or contact events.

Pass: input stays responsive; marks stay attached; held poses do not accidentally freeze the simulation; static props remain still; replay produces the same sequence. Check pause, resume, frame stepping, and variable display rates within admitted fixed workloads.

Scene 4 Physics playground

Begin with existing supported rigid bodies and explicit collision proxies: a ramp, a bowl or bounded play area, several dropped bodies, and one controllable object. Add a controlled squash response as presentation before considering a new solver.

Pass: objects do not tunnel through the agreed test obstacles at the declared maximum speed; reset and replay are repeatable; proxy mismatch is visible in debug mode; contact-driven visual changes do not change collision behavior unexpectedly.

Scene 5 Field stress and correctness

Automate primitive, CSG, transformed, near-tangent, thin-feature, camera-inside, and many-instance cases. Sweep screen coverage and graph complexity independently. Show ray-step count, bounds, normal, object/material ID, collision proxy, and failure overlays.

Pass: invalid bounds or step exhaustion produce explicit diagnostics; no silent geometry loss; performance changes are measurable and traced to a specific workload. This is a test fixture rather than the main showcase.

Optional scene 6 Stamp and cut block

A single bounded block supports add, cut, undo, reset, save, and reload. Start with a capped analytic edit history only if it remains fast; otherwise prototype a small sampled volume with explicit memory, band, precision, and dirty-region rules. Choose one representation for the experiment rather than implementing both fully.

Pass: edits persist and replay; bounds update correctly; undo is bounded in memory; interaction uses the updated geometry under an explicit synchronization rule. Document resolution and lost detail.

Optional scene 7 Deformable body or fluid basin

Choose one first: a small plastically deforming body, or a small viscous-fluid basin. Reuse existing supported physics where possible. If a new particle solver is needed, use an independently validated CPU reference before moving it to compute.

Pass: a fixed workload remains stable within an admitted bounded run; mass/volume error, penetration, particle count, solver cost, and surface-update cost are reported; rendering does not hide instability. Long-duration stability remains unproven and deferred under the current test policy. This scene has its own performance budget and is not a condition for finishing the recommended prototype.

Phased job checklist

Unchecked items below are proposed work. Gate approvals authorize moving to the next phase of the eventual implementation; this document itself does not start that work.

P0 Establish the baseline and approve the design

Dependencies: source audit and selected repository baseline. **Deliverable:** reproducible baseline note and approved examples.

[ ] P0.1 Pin the main commit and relevant cleanup-branch commit; inventory their differences and active ownership before choosing a base.

[ ] P0.2 Run only admitted bounded selectors for SDF, shader, model, ECS, physics, and render-smoke coverage through the production supervisor; record exact commands, platform, failures, and skips. Respect the current 60-second hard ceiling per test command and the quarantined aggregates.

[ ] P0.3 Capture the existing preview and scene3d paths with the same simple model where supported; document unsupported integration points.

[ ] P0.4 Select the reference desktop and a second backend/platform if available; record GPU, driver, OS, resolution, power mode, and build configuration.

[ ] P0.5 Choose Option B as the target or explicitly reduce to A; decide whether the first physics scene uses only existing collision proxies.

[ ] P0.6 Review the three authoring examples and compare syntax options before any grammar change.

[ ] P0.7 Select two clay presets, reference images, camera scale, and proposed performance targets.

[ ] P0.8 Approve the bounded VDB profile, conversion boundary, dependency/licensing review, and resource budgets; identify the DCC roundtrip environment before claiming interoperability.

Gate G0: one chosen baseline, known failures separated from new work, approved scope and syntax direction, and a reproducible test/capture recipe. Do not absorb unfinished self-hosting work merely to unblock the visual experiment.

P1 Define and test the field contract

Dependencies: G0. **Deliverable:** documented supported operations and correctness tests.

[ ] P1.1 Inventory existing primitive, transform, blend, CSG, and sampled-field semantics; extend or replace limits deliberately, with asset migration where an actual used contract changes.

[ ] P1.2 Classify each operation as exact distance, conservative bound, or estimate, including transformed compositions.

[ ] P1.3 Specify bounds, material attribution, rest coordinates, dynamic parameters, and invalid-input behavior.

[ ] P1.4 Add numeric tests for analytic cases and property tests for composition, scale, signs, finite output, and bound containment.

[ ] P1.5 Add adversarial ray cases and a slow reference intersection method for comparison.

[ ] P1.6 Verify parameter-only changes do not trigger unnecessary structural rebuilds or compilation.

[ ] P1.7 Define the CPU-query subset and an explicit unsupported path; test parity within scale-aware tolerances.

Gate G1: every operation allowed in the prototype has documented safety and bounds, with no unexplained missed-surface cases in the agreed corpus. Visual-only estimates cannot enter collision or safe marching by accident.

I1 Add professional field interchange

Dependencies: G1 and P0.8; can run alongside P2-P5. **Deliverable:** bounded OpenVDB import/export, reproducible conversion manifest, one external sculpt asset, and a runtime-storage decision. This is recommended support work, not a request to implement every VDB feature.

[ ] I1.1 Pin the supported OpenVDB release, input/grid profile, tool binaries and hashes, actual dependencies, notices, and supported host platforms. Prove a C-compatible conversion boundary; resolve any new C++ policy exception before coding it.

[ ] I1.2 Define the asset/manifest schema, source retention, explicit grid selection, material binding, units/transforms, band/background values, interpolation and normal rules, bounds, cache version, and unsupported-feature diagnostics.

[ ] I1.3 Build the smallest bounded VDB-to-engine-resource and engine-field-to-VDB path. Enforce the true-distance or re-distancing rule for level-set export. Reuse existing sampled decoding where suitable; add precision, resource or traversal support if required instead of distorting the data to fit an atlas.

[ ] I1.4 Validate sign, zero surface, dimensions, translation/rotation, narrow-band boundaries, gradients and thin features against independent samples. Compare near-band distance values with an independent geometric distance reference, including rebuilt exports. Recompute cache error/Lipschitz/bounds after conversion; test CPU/render agreement and safe traversal at band edges.

[ ] I1.5 Test malformed/truncated files, wrong or absent grid, NaN/Inf, singular/unsupported transforms, unsupported compression/version, path escapes, oversized metadata/topology and allocation/work limits. Fail atomically without publishing partial assets; preserve the previous valid resource.

[ ] I1.6 Run a two-way DCC acceptance case: import an externally authored sphere and thin-feature sculpt; export a sampled Chibi sphere/CSG and reopen it externally. Pin the DCC/tool version and settings, compare geometry numerically, and record expected information loss. A library-only self-roundtrip does not satisfy this gate.

[ ] I1.7 Display the imported sculpt in Scenes 1 and 2 with the same clay material, depth, bounds, picking and shadow checks as analytic models. Keep collision proxies explicit.

[ ] I1.8 Measure a fixed dense-reference versus NanoVDB candidate workload at matched error. Audit C99/GLSL reader integration, buffer limits, shader compiler/backend support, lifetime, upload/rebuild cost and memory before recommending a runtime switch. A recorded decision to keep the simpler cache is valid.

[ ] I1.9 Test deterministic cache regeneration for pinned tools and settings, stale-cache invalidation, content hashes, reload, and export provenance; retain original source and conversion recipe.

[ ] I1.10 Publish the supported-profile table, actual import/export commands, bounded test results, dependency notices and limitations. Record USD, quantized NanoVDB variants, animation sequences and mutable sparse topology as explicit later decisions.

Gate GI1: the supported VDB profile completes the external two-way roundtrip within declared voxel-relative tolerances, imports safely, and renders correctly in the mixed scene. Source preservation and regeneration work, and resource costs fit the approved limits. If suitable DCC access is unavailable, the DCC gate stays blocked; do not substitute an unverified compatibility claim. SideFX's VDB From Polygons is a useful reference for signed/unsigned/fog selection and voxel-resolution limits, not evidence that this Chibi workflow has already run.

P2 Integrate the 3D scene path

Dependencies: G1 for safe rendering; material experiments can proceed in parallel in the existing preview. **Deliverable:** mixed opaque 3D proof scene.

[ ] P2.1 Connect one existing SDF model to scene3d using the smallest compatible rendering route.

[ ] P2.2 Validate camera rays, projection, viewport, transforms, clipping, and camera-inside behavior.

[ ] P2.3 Validate ordinary-geometry/SDF depth in both draw orders and at intersections.

[ ] P2.4 Preserve material/part IDs and surface coordinates. Compare a mesh control and SDF control with identical graph parameters under matched normals, coordinates, lights, and exposure; both must use the same scene3d material response.

[ ] P2.5 Add bounded instance extraction, culling, resource reuse, and lifecycle cleanup.

[ ] P2.6 Add one shadowed key light and picking; keep expensive options disabled until measured.

[ ] P2.7 Add regression captures for the existing preview, 2.5D/material path, and scene3d.

Gate G2: Scene 2 passes depth and camera tests, mesh/SDF controls agree under matched shading conditions, picking hits the displayed surface, and the SDF casts and receives the chosen directional shadow. The shadow silhouette must match the field rather than its proxy box, including off-screen casters and animated bounds. Other shadow types and GI remain unsupported until integrated; do not silently treat a manual primitive as true SDF occlusion geometry. A second backend either passes or is explicitly outside the prototype scope with a clear blocker.

P3 Build the clay material and look controls

Dependencies: G0 for reference direction, existing preview for initial work, G2 for final scene validation. **Deliverable:** Scene 1 and saved presets.

[ ] P3.1 Reuse the existing GGX material with Metallic=0 and existing Bump/Normal Map nodes. Verify color/exposure/roughness semantics and keep the first Oiliness control an explicit mapping to supported settings.

[ ] P3.2 Add authored low-frequency form irregularity independently from microdetail.

[ ] P3.3 Add physically scaled fingerprints and scuffs in normal and roughness response.

[ ] P3.4 Verify rigid and deformed coordinate attachment, including transformed normal bases.

[ ] P3.5 Decide whether the existing single-lobe response has a demonstrated visual deficiency. Only then approve and compare a second oily reflection lobe under identical lighting.

[ ] P3.6 Evaluate optional thickness-aware scattering only if it improves the selected reference match enough to justify its cost.

[ ] P3.7 Expose readable controls, reset defaults, preset persistence, and isolated A/B toggles.

[ ] P3.8 Review close-up, gameplay-distance, silhouette, grazing-light, and moving-camera captures.

Gate G3: approved visual preset with reproducible parameters, stable detail, and measured costs for each implemented optional effect. Rejudge the final preset under scene3d lighting and fixed exposure; preview lighting is not evidence of matching world-space response. Extra reflection lobes and scattering may remain unimplemented without blocking this gate. No claim that one roughness value is universally correct.

P4 Add stepped presentation and ECS behavior

Dependencies: G2 and basic G3 material. **Deliverable:** Scene 3 plus timing and lifecycle tests.

[ ] P4.1 Separate simulation time, presentation pose index, and material-event time.

[ ] P4.2 Implement deterministic per-instance cadence and phase with smooth root motion as the initial default.

[ ] P4.3 Keep camera and input responsive while pose/deformation is held.

[ ] P4.4 Trigger a finite set of authored local mark, material, or shape-parameter states from bounded events. Accumulated persistent sculpt edits remain in P8.

[ ] P4.5 Define previous-pose data and antialiasing/history reset behavior for held-pose transitions.

[ ] P4.6 Test 30, 60, and 120 Hz display/presentation conditions, pauses, seeks, frame stepping, and reset.

[ ] P4.7 Test spawn/despawn, shared assets, hot reload, and parameter changes without resource leaks.

Gate G4: the same recorded input yields the intended pose sequence; untouched surfaces remain coherent; no frame-rate-dependent gameplay or persistent ghosting.

P5 Add the physics playground

Dependencies: G2, existing physics baseline, and G4 for contact presentation. **Deliverable:** Scene 4 and proxy policy.

[ ] P5.1 Choose supported colliders for the first bodies and document their relationship to rendered shapes.

[ ] P5.2 Define maximum speeds, smallest obstacles, fixed-step/substep policy, and acceptable contact tolerance.

[ ] P5.3 Exercise drops, ramps, stacking where supported, and a controllable body against the same test fixtures.

[ ] P5.4 Add collision-proxy and contact-normal overlays.

[ ] P5.5 Connect contact events to bounded visual compression and a finite set of authored local surface states; do not introduce persistent edit history here.

[ ] P5.6 If field collisions are added, test broad phase, narrow phase, response, friction, and continuous/swept behavior separately.

[ ] P5.7 Verify no synchronous GPU readback is required in the normal input/physics loop.

Gate G5: stable declared interactions with no unexplained tunneling or severe render/collider disagreement in the agreed speed and size envelope. General deformable collision is not implied by this gate.

P6 Profile and finish the vertical slice

Dependencies: G2-G5 and GI1 for the complete recommended scope. **Deliverable:** four showcase scenes including the imported asset, stress fixture, captures, performance report, and documented launch/reset path. A visual-only checkpoint may precede GI1 but must not be described as completed professional interchange.

[ ] P6.1 Profile CPU frame preparation, GPU intersection, material/shadow work, physics, memory, and compilation separately.

[ ] P6.2 Sweep field complexity, visible instances, screen coverage, and optional material features independently.

[ ] P6.3 Remove unnecessary rebuilds and allocations before introducing a new acceleration structure.

[ ] P6.4 Measure bounds/culling, static code reuse, specialization, step controls, and any reduced-resolution mode.

[ ] P6.5 Test grazing geometry, thin features, animated bounds, and history rejection after each optimization.

[ ] P6.6 Record warm and cold iteration times. Failed edits must preserve the last valid runtime generation with visible stale-preview diagnostics, publish no partial resources, recover after correction, and reject export/preparation from invalid current source.

[ ] P6.7 Produce a short interactive walkthrough and matched still/video comparisons, with exact settings and build identity.

[ ] P6.8 Run independent code review, relevant admitted bounded regressions, backend smoke checks, and fixed-count reset/reload selectors. New selectors need explicit admission. Keep quarantined aggregates excluded; long-duration stability evidence remains deferred.

Gate G6: the declared quality target fits the approved workload on the named machine, required tests pass, known limits are visible, and the demo can be reproduced from its instructions. Failing a budget means simplifying, fixing, or changing the target explicitly, not silently lowering quality.

P7 Prove the shader language bridge

Dependencies: stable G1 semantics, G6 evidence, and a separately approved language design. **Deliverable:** one material expression and one compiler-known analytic field operation through a typed Chibi-facing frontend, retaining the field bounds and safety contract.

[ ] P7.1 Approve expression entry signatures, float32 types, existing parameter/image semantics, pure helper rules, and failure behavior; defer new resource models.

[ ] P7.2 Map the frontend into existing typed shader/model representations instead of adding an unrelated compiler stack.

[ ] P7.3 Preserve source spans through lowering and generated shader compilation.

[ ] P7.4 Verify uniform/resource reflection and deterministic cache keys.

[ ] P7.5 Compare generated output and runtime images with the equivalent graph-authored versions.

[ ] P7.6 Test rejection of host-only features, unsupported stages, loops, control flow, recursion, and new resource models; test accepted precision and backend behavior.

[ ] P7.7 Measure compile latency, code size, runtime cost, and diagnostics before broadening the subset.

Gate G7: the bridge is useful and semantically consistent for the two examples. Expanding it to a general shader language is a new scoped decision; it does not block completing the clay demonstration.

P8 Optional persistent sculpting or simulation

Dependencies: G6 and explicit choice of C or D. **Deliverable:** one tightly bounded research scene.

[ ] P8.1 Select the interaction first: stamp/cut, permanent body dent, or fluid flow.

[ ] P8.2 Choose the representation and write down its precision, memory, topology, and collision consequences.

[ ] P8.3 Build a tiny correctness reference and a fixed workload before GPU optimization.

[ ] P8.4 Define edit/solver ordering, dirty-region updates, and CPU/GPU synchronization.

[ ] P8.5 Add bounded history or snapshots, reset, and serialization where edits persist.

[ ] P8.6 Measure simulation and surface-generation cost independently from rendering.

[ ] P8.7 Evaluate numerical drift, detail loss, and contact quality within admitted fixed workloads; record long-run stability as deferred evidence.

[ ] P8.8 Decide whether the result merits an engine feature, remains a demo, or should be stopped.

Gate G8: the selected interaction is demonstrably useful within its declared limits. Avoid adding the other two interactions until this decision is made.

Proposed performance and quality targets

These targets are a starting proposal for a named reference desktop selected in P0. They are not claims about present Chibi performance. Record a second, lower-cost quality tier only after the baseline is measured.

Metric	Initial target	Measurement rule
Presentation	1280 by 720 internal resolution at 60 Hz	Fixed camera paths and explicit scaling settings
End-to-end frame time	95th percentile at or below 16.7 ms	Warm run; also report median, 99th percentile, and worst stutter
GPU frame	At or below 12 ms at the target workload	Timestamp supported passes; report unsupported timing explicitly
SDF intersection plus clay shading and shadows	At or below 8 ms of GPU frame	Report each component; no hidden resolution reduction
Main-thread simulation and frame preparation	At or below 4 ms	Separate compilation, loading, and rendering submission
Presentation cadence	12 poses per second by default	Simulation and input remain independent
Parameter edit feedback	Within the next two presented frames when no recompile is needed	Test warmed resources; distinguish topology edits
Small graph/material rebuild	Target under 500 ms warm; report cold time separately	Same graph, build, machine, and cache state
Resource lifetime	No unexplained net growth across a fixed admitted reset/reload sample	Fixed iteration count within the supervisor ceiling; long-duration behavior remains unproven
Prototype-owned GPU memory	Provisional 256 MiB ceiling	Include field/material resources and temporary targets; exclude and report shared engine baseline

Begin load testing with 1, 4, 8, and 16 visible instances, with 1, 8, 16, and 32 shape operations per resource where supported. Treat these as test points, not an already supported capacity guarantee. Select the shipping demo cap from measured results. Test 25%, 50%, and near-full screen coverage because a small object count can still be expensive when every pixel marches through it.

As an initial quality experiment, compare primary-ray caps of 64, 96, and 128 steps and a lower-cost shadow configuration. Choose defaults from the worst valid scene, not the easiest sphere. Expose step exhaustion in the diagnostic view. Scale hit and normal tolerances to world units and pixel footprint, with bounded minimum and maximum values.

Proposed bounded benchmark sample: up to 5 seconds of warmup and 20 seconds of the same camera/input sequence, with a fixed frame/workload cap and an overall command deadline below the supervisor's 60-second ceiling. A new benchmark selector must be explicitly admitted before use.

Record sample size and uncertainty; do not reconstruct a prohibited sustained run from chunks. Long-duration stability and rare-stutter evidence remain deferred. Track cold-start and shader-compilation hitches separately. Disable frame caps only for relevant throughput tests and restore the actual presentation settings for perceptual review.

Optional volume memory experiment

For a future sampled field, a dense 128-cubed R16 field is 4 MiB before mips; 256-cubed is 32 MiB. A full 3D mip chain approaches an 8/7 multiplier. Double buffering a 256-cubed field therefore approaches 73.1 MiB before materials, edit history, scratch space, or acceleration data. These are calculated storage examples, not a recommendation to copy Claybook's world dimensions. Establish units, narrow-band range, quantization error, gradient quality, and update cost before choosing a format.

Options and tradeoffs for later work

Analytic graph or sampled volume

Analytic graphs are compact and expressive for reusable shapes and parameter animation. Evaluation cost rises with the active operation graph; permanent edits can accumulate expensive history. Sampled volumes make bounded local edits and texture sampling attractive, but introduce resolution limits, memory, interpolation errors, band maintenance, and update scheduling. A hybrid can use analytic authoring plus cached local volumes, but that is an additional system to validate.

Prefer analytic authoring for the procedural playground and the I1 sampled-field bridge for imported sculpts. Extend either where needed. A mutable volume experiment is separately justified by retained sculpting or a solver-generated surface; importing a static sampled asset does not require that experiment. The JCGT study of SDF grids is useful for understanding traversal, interpolation, and normal reconstruction; its hardware and path-tracing conclusions should not be treated as an automatic fit for Chibi.

Raymarching or extracted mesh

Raymarching keeps procedural surfaces directly editable and avoids tessellation management, but costs screen-space intersection work and requires careful depth, shadow, and temporal integration. Extracted meshes use the ordinary geometry pipeline and may fit existing deformation tools better, but add meshing, topology, normal, update, and memory work. Neither is universally superior. Compare the two only after a specific animated or editable workload exposes a problem in the baseline.

Visual deformation or physical deformation

A squash curve can communicate weight at low cost, but it is authored presentation. Elastic or plastic simulation needs a solver, constraints or constitutive behavior, contact handling, and stable state. XPBD is worth considering for a small constraint-based experiment because it addresses the time-step and iteration dependence of stiffness in ordinary PBD; it does not supply a complete plastic-clay model by itself. See the XPBD paper. A fluid method is another separate choice; Position Based Fluids is a candidate reference, not a claim about Claybook's SPH implementation.

Broad portability or compute-first research

The baseline should fit the supported Chibi/Cute path. GPU compute, sparse volumes, indirect dispatch, advanced resource barriers, and hardware ray tracing should be introduced only after capability checks and a demonstrated need. Keep a fallback or explicitly declare the experiment's platform limit. A successful desktop research result does not prove the same feature works on every backend.

Review and completion package

Use small reviewable changes along the phase boundaries. Current testing contracts require the production supervisor and exact admitted selectors, with a 60-second hard ceiling per test command; full render/native/Studio aggregates remain quarantined. New selectors need explicit admission. Long stress captures remain deferred; do not reconstruct them from chunks. Keep renderer integration, material appearance, timing/ECS behavior, physics queries, and compiler-language work independently reviewable where possible. Merge sequencing must follow the actual dependencies and current branch state.

The prototype is ready to evaluate when it includes:

- Four working showcase scenes and the stress/correctness fixture.
- A concise launch, controls, reset, preset, and replay guide.
- A source-pinned baseline and a list of actual changes.
- Test results with commands, platform details, skips, and remaining limits.
- Matched images and a short capture comparing baseline and chosen clay settings.
- CPU/GPU/memory/compile measurements, workload definitions, and quality settings.
- A supported-operation table, collision policy, coordinate policy, and shader-language design decision.
- A verified VDB import/export profile, source-preserving cache workflow, external roundtrip evidence, and dependency/license manifest.
- A measured runtime-storage decision, plus explicitly scoped NanoVDB/USD follow-ons where justified.

- A clear recommendation to continue, reduce scope, or stop each optional research direction.

The revised recommendation is a four-scene playground with whatever bounded SDF extensions it requires, plus a professional OpenVDB asset bridge. Review syntax and conversion-boundary choices before implementation. NanoVDB runtime adoption, USD scene integration, persistent sculpting, true plastic deformation, and fluids retain their own evidence-based gates.

Reference gallery and reading order

All links below were opened or resolved through a primary source on 19 October 2026. Video entry pages are distinguished from watched content. Third-party screenshots and images are reference material; their availability is not a license to redistribute them as Chibi assets.

Start with the look

1. **Lindsey Howell, Aardman and Bath production thesis.** PDF. Read chapter 4, printed pages 67-74 and 83-85. Inspect figures 4-7, 4-10, 4-11, and 4-12 for material ranges, textured comparisons, and worked surfaces. Strongest production-oriented reference; the implementation is not a ready-made real-time Chibi shader.
2. **Creating a Realistic Clay Shader for Digital Stop Motion.** PDF. Pages 4-6 contain material comparisons and the final result. Useful compact visual checklist; it also records a grazing-angle mismatch.
3. **Jonas Johansson, Clay Material in Unity.** Article and animated comparisons, direct clay material image. The static image was inspected. The article's embedded video was not watched. Use the image to judge fingerprint scale and roughness, and compare its deliberate surface-boil approach with the quieter proposal here.
4. **Mike Dietz, Our Animation Process.** First-hand Skullmonkeys production article and photographs. Useful for thinking about short gameplay actions and testing timing before expensive finishing. This describes photographed stop-motion production, not a real-time SDF technique.

Understand the Claybook architecture

5. **Sebastian Aaltonen, GPU-based clay simulation and ray-tracing tech in Claybook.** GDC 2018 slides, official session and video entry. PDF read; selected slides visually checked; video not watched. Use pages 7-9 for representation, 19-24 for updates/traversal, 44-54 for bodies and fluids, and 58 for integration. This is a GDC reference, not a SIGGRAPH talk.
6. **Claybook publisher page.** Steam page and media. Use it for the overall scale, colorful miniature presentation, and interaction ambition. Store media was not treated as evidence of implementation details. The GDC slides also link an official trailer; its destination did not load in this review, so it is a source-linked viewing suggestion rather than a watched reference.

Keep the mathematics and simulation honest

7. **John C Hart, Sphere Tracing.** Research paper PDF, university publication record. Sections 2.1-2.2 and the appendices explain distance bounds and transformations. Read before adding arbitrary displacement or deformation.
8. **Hansson-Soderlund, Evans, and Akenine-Moller, Ray Tracing of Signed Distance Function Grids.** Journal page, small PDF. Read for sampled-grid intersection and normal behavior. Results are workload and hardware specific.

9. **Muller and colleagues, Position Based Dynamics.** Original paper. Background for constraint projection and the difference between a field representation and a solver.
10. **Macklin, Muller, and Chentanez, XPBD.** Paper. Candidate for a bounded deformable-body experiment; plasticity, collision, and visual surface tracking still require design.
11. **Macklin and Muller, Position Based Fluids.** Paper. Optional fluid research, with its own constraints, neighbor search, and rendering work.
12. **Matthias Muller, Ten Minute Physics.** Author's video, demo, code, and notes index. Episodes 9-12, 16, and 20 are useful small-scope learning examples. Index inspected; videos not watched for this plan. Useful inspiration for short self-contained Chibi physics scenes rather than a requirement to adopt a new solver.

Chibi implementation reading map

13. **Model construction and compilation.** model_compile.c, model_nodes.c, model_program.h. Follow shape semantics, bounds, parameters, hit/part resolution, and dependencies.
14. **Rendering integration.** sdf_material.frag, mesh_surface.c, scene_renderer.c, scene3d.c. The key boundary is between the existing SDF material path and free-camera world-space rendering.
15. **Shader backend.** shader_adapter.c, program.h, program.c, Cute compilation connection. Reuse this path for the future frontend.
16. **Language and expression boundaries.** grammar.js, math_expression.c, language reference. Distinguish full CPU authoring from the current shader expression subset.
17. **ECS and physics.** ecs.h, Collider3D contract, physics3d.c. No new allocator requirement or implied SDF collider should be introduced.
18. **Testing and dependencies.** testing.md, dependencies.cmake. Cute is pinned with local patches; use the actual backend rather than assuming upstream capabilities.

Professional field and scene interchange

19. **OpenVDB data model and build profile.** Overview, build guide, pinned 13.0.0 core CMake, license, vdb_tool. Primary documentation and source inspected. Use for the bounded level-set interchange contract and dependency decision.
20. **NanoVDB runtime representation.** FAQ, pinned PNanoVDB header. Static topology, contiguous layout, portable readers; benchmark the actual Chibi workload before adoption.
21. **OpenUSD volume scene references.** UsdVol overview, OpenVDBAsset API, 26.08 license and notices. Scene references and grid selection do not establish level-set rendering or Chibi material interchange.
22. **Mesh and DCC bridges.** Khronos glTF 2.0, SideFX VDB From Polygons, existing Chibi mesh-to-SDF tool, existing glTF import. Read for the explicit mesh/field conversion boundary and real external acceptance fixtures.